

Word Vectorization

Contents:

1. Word Vectorization
2. One-hot encoding
3. Bag of words (BOW)
4. Term Frequency- Inverse Document Frequency (TFIDF)
5. Word2Vec
6. GloVe
7. BERT
8. Cosine Similarity

Vectorization

- Words within a **text document are transformed into importance numbers** by a text vectorization process.
- There are many different text vectorization scoring schemes: **One-hot encoding, TFIDF, GloVe, Word2Vec, Bag-of-words, BERT**, etc

1. One-hot encoding

- One-hot encoding is the process of turning categorical factors into a numerical structure that machine learning algorithms can readily process.
- It functions by representing each category in a feature as a binary vector of 1s and 0s, with the vector's size equivalent to the number of potential categories.

Index	Animal	One-Hot code					
0	Dog		0	1	0	0	0
1	Cat		1	0	1	0	0
2	Sheep		2	0	0	1	0
3	Horse		3	0	0	0	1
4	Lion		4	0	0	0	1

- For example, if we have a feature with three categories (A, B, and C), each category can be represented as a binary vector of length three, with:
 - the vector for category A being $[1, 0, 0]$,
 - the vector for category B being $[0, 1, 0]$, and
 - the vector for category C being $[0, 0, 1]$.

Python code:

```
import numpy as np

# Define the corpus of text
corpus = [
    "The quick brown fox jumped over the lazy dog.",
    "She sells seashells by the seashore.",
    "Peter Piper picked a peck of pickled peppers."
]

# Create a set of unique words in the corpus
unique_words = set()
for sentence in corpus:
    for word in sentence.split():
        unique_words.add(word.lower())

# Create a dictionary to map each
# unique word to an index
word_to_index = {}
for i, word in enumerate(unique_words):
    word_to_index[word] = i
```

```
# Create one-hot encoded vectors for
# each word in the corpus
one_hot_vectors = []
for sentence in corpus:
    sentence_vectors = []
    for word in sentence.split():
        vector = np.zeros(len(unique_words))
        vector[word_to_index[word.lower()]] = 1
        sentence_vectors.append(vector)
    one_hot_vectors.append(sentence_vectors)

# Print the one-hot encoded vectors
# for the first sentence
print("One-hot encoded vectors for the first sentence:")
for vector in one_hot_vectors[0]:
    print(vector)
```

Output:

One-hot encoded vectors for the first sentence:

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

2. Bag of words (BOW)

- Bag of Words (BoW) simply counts the frequency of words in a document.
- Thus the vector for a document has the frequency of each word in the corpus for that document.
- Consider two documents:

- Consider two documents:
 1. John likes to watch movies. Mary likes movies too.
 2. Mary also likes to watch football games.
- Based on these two text documents, a list is constructed as follows for each document.

"John", "likes", "to", "watch", "movies", "Mary", "likes", "movies", "too"

"Mary", "also", "likes", "to", "watch", "football", "games"

- The Bag-of-words for these documents is:

```
BoW1 = {"John":1, "likes":2, "to":1, "watch":1, "movies":2, "Mary":1, "too":1};  
BoW2 = {"Mary":1, "also":1, "likes":1, "to":1, "watch":1, "football":1, "games":1};
```

3. TFIDF

- TF-IDF is one of the most common.
- TF-IDF is useful in many natural language processing applications.
- For example, Search Engines use TF-IDF to rank the relevance of a document for a query. TF-IDF is also employed in text classification, text summarization, and topic modeling.

Term Frequency:

- **Term Frequency (TF)** of a term or word is the **number of times the term appears in a document compared to the total number of words** in the document.

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

- Consider the term ***apple*** appears 20 times in a document that contains a total of 100 words. Calculate the TF of the word *apple*.

Solution:

$$TF_{\text{apple}} = 20 / 100 = 0.2$$

Inverse Document Frequency:

- Inverse Document Frequency (IDF) of a term reflects the proportion of documents in the corpus that contain the term.
- Words unique to a small percentage of documents receive higher importance values than words common across all documents (e.g., a, the, and).

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

- To avoid divide by zero, 1 can be added to the denominator as below:

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term} + 1}\right)$$

TF-IDF

- The TF-IDF of a term is calculated by multiplying TF and IDF scores.

$$TF-IDF = TF * IDF$$

Numerical:

- Compute the TF-IDF of the word ***apple***.

Solution:

$$TF-IDF = 0.2 * 2 = 0.4$$

Numerical

Given below documents, compute the TF-IDF.

- Doc1: 'The car is driven on the road' ,
- Doc2: 'The truck is driven on the highway'

Solution:

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2) = 0$	0	0
Car	1/7	0	$\log(2/1) = 0.3$	0.043	0
Truck	0	1/7	$\log(2/1) = 0.3$	0	0.043
Is	1/7	1/7	$\log(2/2) = 0$	0	0
Driven	1/7	1/7	$\log(2/2) = 0$	0	0
On	1/7	1/7	$\log(2/2) = 0$	0	0
The	1/7	1/7	$\log(2/2) = 0$	0	0
Road	1/7	0	$\log(2/1) = 0.3$	0.043	0
Highway	0	1/7	$\log(2/1) = 0.3$	0	0.043

Python code:

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    'The car is driven on the road',
    'The truck is driven on the highway',
]

vectorizer = TfidfVectorizer()

# TD-IDF Matrix
X = vectorizer.fit_transform(corpus)

# extracting feature names
tfidf_tokens = vectorizer.get_feature_names_out()
tfidf_tokens
```

Output:

```
array(['car', 'driven', 'highway', 'is', 'on', 'road', 'the', 'truck'],
      dtype=object)
```

```
import pandas as pd

result = pd.DataFrame(
    data=X.toarray(),
    index=["Doc1", "Doc2"],
    columns=tfidf_tokens
)
```

result

Output:

	car	driven	highway	is	on	road	the	truck
Doc1	0.424717	0.30219	0.000000	0.30219	0.30219	0.424717	0.60438	0.000000
Doc2	0.000000	0.30219	0.424717	0.30219	0.30219	0.000000	0.60438	0.424717

Cosine similarity :

- **calculates the cosine of the angle** between two documents
- **0 indicates exactly opposite, 1 indicates exactly same**, and while others are intermediate values

$$\text{Cosine similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- **Where,**
- **A= TFIDF of document A**
- **B=TFIDF of document B**

4. Word2Vec

- Word2Vec is an algorithm that uses shallow 2-layer, not deep, neural networks to ingest a corpus and produce sets of vectors.
- Some key differences between TF-IDF and word2vec is that
 - TF-IDF is a statistical measure that we can apply to terms in a document and then use that to form a vector whereas
 - word2vec will produce a vector for a term and then more work may need to be done to convert that set of vectors into a singular vector or other format.
 - Additionally TF-IDF does not take into consideration the context of the words in the corpus whereas word2vec does.

4. GloVeVector

- GloVe, or Global Vectors for Word Representation, is an unsupervised learning algorithm that obtains vector word representations by analyzing the co-occurrence statistics of words in a text corpus.
- These word vectors capture the semantic meaning and relationships between word.
- To start with GloVe, first we have to download the pre-trained model hosted at <https://nlp.stanford.edu/projects/glove/>; there are a total four pre-trained models available; you can choose anyone you want.
 - glove.6B.50d.txt
 - glove.6B.100d.txt
 - glove.6B.200d.txt
 - glove.6B.300d.txt

Word similarity with GloVeVector

```
In [6]: word = 'cat'
other = ['dog', 'bike', 'kitten', 'puppy', 'kite', 'computer', 'neuron']
for w in other:
    dist = torch.norm(glove[word] - glove[w]) # euclidean distance
    print(w, float(dist))
```

```
dog 2.6811304092407227
bike 6.022563934326172
kitten 4.454165935516357
puppy 3.9275598526000977
kite 5.859299659729004
computer 6.960631847381592
neuron 7.568033218383789
```

Finding closest word with GloVeVector

```
In [8]: print_closest_words(glove['nurse'])
```

```
doctor 3.818501  
nurses 3.915796  
dentist 4.050089  
midwife 4.1329455  
physician 4.172601
```

```
In [9]: print_closest_words(glove['computer'])
```

```
computers 3.1775317  
software 3.7114491  
technology 4.4087934  
hardware 4.5274196  
pc 4.581429
```

5. BERT

- Bidirectional Encoder Representations from Transformers
- BERT is an ML/NLP technique developed by Google that uses a transformer based ML model to convert phrases, words, etc into vectors.
- Key differences between TF-IDF and BERT are as follows:
 - TF-IDF does not take into account the semantic meaning or context of the words whereas BERT does.
 - Also BERT uses deep neural networks as part of its architecture, meaning that it can be much more computationally expensive than TF-IDF which has no such requirements.

End of Topic